アジャイルワーク2

ニューラルネットワーク
重みデータ圧縮に関する実験計画書

学籍番号　25G1017
氏名　岩澤大心

作成日：2026年10月9日

改訂日：2026年10月16日

提出期限：2026年10月16日（金）

目次

1 実験の目的 **1**
1.1 データ圧縮が必要な理由 1
1.2 検証する仮説 1

2 実験の理論 **1**
2.1 圧縮方式の調査と選択 1
2.2 ネットワークのデータ量 1
2.3 閾値による枝刈り 3
2.4 疎行列の格納方式 3
2.5 評価指標 4

3 実験方法 **5**
3.1 実験条件 5
3.2 学習・検証・最終評価データの分離 6
3.3 実験条件の一覧 7
3.4 プログラムの実装手順 8
3.5 性能確認方法 8
3.6 閾値の選択とCSRの採用条件 9
3.7 結果の記録様式 10

1 実験の目的

本実験の目的は，カラーチャートの読み取り値を補正するニューラルネットワークについて，Arduino上で使用する重みデータを圧縮し，限られたメモリ内で推論精度を保ったまま 隠れ層の次元数を拡大できる条件を明らかにすることである．

1.1 データ圧縮が必要な理由

対象ネットワークは入力3次元，隠れ層2層，出力3次元である．授業資料では，隠れ層60次元は コンパイル・実行が可能である一方，70次元ではメモリ不足によるコンパイルエラーが確認されている． 隠れ層を大きくすると表現能力の向上が期待できるが，第2層の重み数は次元数の二乗で増加するため， 32 bit浮動小数点数のまま全要素を保存する方法には限界がある．

したがって，Arduinoの限られたメモリに，より大きな隠れ層を持つモデルを搭載するには， 推論への影響が小さい情報を削減し，重みデータを小さくする必要がある．ただし，容量だけを 小さくしても補正精度や実行速度が大きく悪化すれば実用にならないため，データ量，精度，速度を 同時に測定し，圧縮の有効性を判断する必要がある．

そこで，L1正則化によって絶対値が小さくなった重みを0へ置き換える枝刈りと， 非ゼロ要素だけを保持するCSR（Compressed Sparse Row）形式を組み合わせた方式を採用する． 隠れ層40，50，60次元において，圧縮前のDense-FP32と枝刈り+CSRについて， モデル容量，Arduinoへの搭載可否，推論時間，平均二乗誤差（MSE）を同一条件で比較し， メモリ削減量と推論品質のトレードオフを定量化する．

1.2 検証する仮説

1. L1正則化後の絶対値が小さい重みは推論への寄与が小さいため，適切な閾値で0化してもMSEの増加は小さい．
2. 十分な疎性が得られれば，CSR形式は列番号等の付加情報を含めても密行列より小さくなる．

2 実験の理論

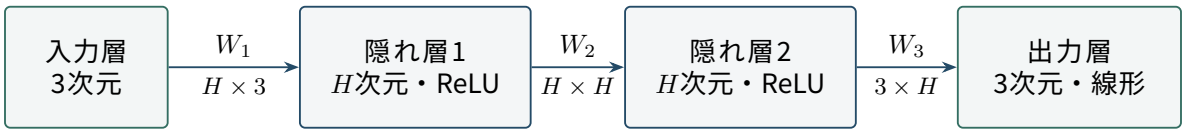
2.1 圧縮方式の調査と選択

本実験では，重要度の低い接続を除去する枝刈りの研究[2]を参考に， 絶対値の小さい重みを0へ置き換える．L1正則化による係数縮小[1]を利用し， 検証用データで誤差を確認しながら枝刈り閾値を選ぶ． ただし，0に置き換えただけのFP32密配列では要素数が変わらず，格納容量は減らない． そこで，非ゼロ要素と位置情報を保存する疎行列形式を組み合わせる．

格納形式はCOOとCSRについて，値と索引の容量，行列ベクトル積の実装のしやすさを比較する． CSRは行ごとに非ゼロ要素を取り出せるため，出力ニューロン単位の積和演算に対応させやすい． このため，実装・評価する圧縮方式は枝刈り+CSRに絞り，圧縮前のDense-FP32を比較基準とする． 容量，MSE，推論時間から，この方式が有効となる閾値と隠れ層次元を明らかにする．

2.2 ネットワークのデータ量

隠れ層の次元数をHとすると，各層の重み行列は $H \times 3$，$H \times H$，$3 \times H$である．ネットワーク構成を図1に示す．



$H = 40, 50, 60$で圧縮前後を比較する

図 1: 対象ネットワークの構成．各層でバイアスを加算し，隠れ層だけにReLUを適用する．

順伝播は

$$\boldsymbol{h}_1 = \mathrm{ReLU}(W_1 \boldsymbol{x} + \boldsymbol{b}_1), \tag{1}$$
$$\boldsymbol{h}_2 = \mathrm{ReLU}(W_2 \boldsymbol{h}_1 + \boldsymbol{b}_2), \tag{2}$$
$$\boldsymbol{y} = W_3 \boldsymbol{h}_2 + \boldsymbol{b}_3 \tag{3}$$

によって計算される．パラメータ数は

$$N_{\mathrm{weight}} = H^2 + 6H, \tag{4}$$
$$N_{\mathrm{bias}} = 2H + 3, \tag{5}$$
$$N_{\mathrm{total}} = H^2 + 8H + 3 \tag{6}$$

となる．すべてを`float`（1要素4 byte）で保持した場合のデータ量は $B_{\mathrm{dense}} = 4N_{\mathrm{total}}$ byteである．理論値を表1に示す.

表 1: 密形式で保持した場合のパラメータ量（配列管理情報を除く）

H	重み数	バイアス数	合計要素数	FP32データ量 [byte]
40	1,840	83	1,923	7,692
50	2,800	103	2,903	11,612
60	3,960	123	4,083	16,332
70	5,320	143	5,463	21,852
100	10,600	203	10,803	43,212
200	41,200	403	41,603	166,412

この値以外にも，プログラム本体，入出力用配列，スタック等がArduinoのメモリを使用する． 本実験では隠れ層出力をFP32配列2本で保持する設計とし，その作業領域は $8H$ byte（$H = 60$では480 byte）となる．さらに，6×6画素のRGB値を 各成分1 byteで保持する場合，画像配列だけで108 byteを使用する． ビルド結果に表示されるRAM使用量は通常，グローバル変数等の静的領域であり， ローカル配列を含むスタックや動的確保領域をすべて含むとは限らない． フラッシュ使用量，静的RAM使用量，推論用作業配列の容量を区別して記録し， 実機での連続推論も確認して搭載可否を判断する．

圧縮前後とも重み配列とバイアスを読み取り専用として扱い，宣言の修飾子と配置方針を統一する． ただし，`const`を付けた配列の配置先と読出方法はボード・ツールチェーンに依存するため， 必ずフラッシュへ配置されるとは仮定しない．実験開始時に対象ボードの配置を確認し， 必要な読出方法を圧縮前後でそろえる．モデル容量は重み，バイアス，CSR索引の 各配列の`sizeof`の合計で求め，配置やパディングを含むビルド全体の使用量と区別する．

2.3 閾値による枝刈り

学習後の重みwに閾値τを適用し，

$$\widetilde{w} = \begin{cases} 0 & (|w| < \tau), \\ w & (|w| \geq \tau) \end{cases} \tag{7}$$

とする．0となった割合（疎性）Sは，全重み数をN，非ゼロ重み数をKとして

$$S = 1 - \frac{K}{N} \tag{8}$$

で求める．本実験では$\tau = 0$を非圧縮の基準とし，$\tau \in \{0.001, 0.005, 0.01, 0.02, 0.05\}$を候補にする． 0.01は候補値の一つであり，固定値とはせず，MSEと容量の両方から選ぶ．

バイアスは要素数が少なく，0化による削減効果が小さいため，FP32のまま保持する． これにより，圧縮による変化を重み行列に限定する．

2.4 疎行列の格納方式

$m \times n$行列にK個の非ゼロ要素がある場合，COO形式は値，行番号，列番号を列挙する． 一方，CSR形式は値配列，列番号配列，各行の開始位置を表す行ポインタ配列を使用する．CSRは疎行列ベクトル積で広く用いられる格納形式である[3]． $H \leq 200$では行・列番号を`uint8_t`，行ポインタを`uint16_t`で表現できるため， 理論データ量は概ね

$$B_{\mathrm{COO}} = (4 + 1 + 1)K = 6K, \tag{9}$$

$$B_{\mathrm{CSR}} = (4 + 1)K + 2(m + 1) \tag{10}$$

となる．3層分のCSR配列とFP32バイアスを合計すると

$$B_{\mathrm{CSR,total}} = 5K + 12H + 24 \tag{11}$$

byteである．したがってCSRが密形式より小さくなる目安は

$$K < \frac{4H^2 + 20H - 12}{5} \tag{12}$$

となる．CSRでは行番号を要素ごとに保存しないため，本実験のような行列積に適している．上式のKは3層分の非ゼロ重み数の合計である． 配列間のアラインメント等はビルド全体の使用量に影響し得るため，最終的には 各配列の`sizeof`の合計とビルド結果の両方で確認する．CSRが密形式より大きくなる条件では採用しない．

枝刈りとCSRへの格納の対応を図2に示す． 図は説明用の3×4行列であり，実験で使用する学習済み重みではない． 非ゼロ値は行優先で並べ，列番号と行ポインタの添字は0始まりとする．

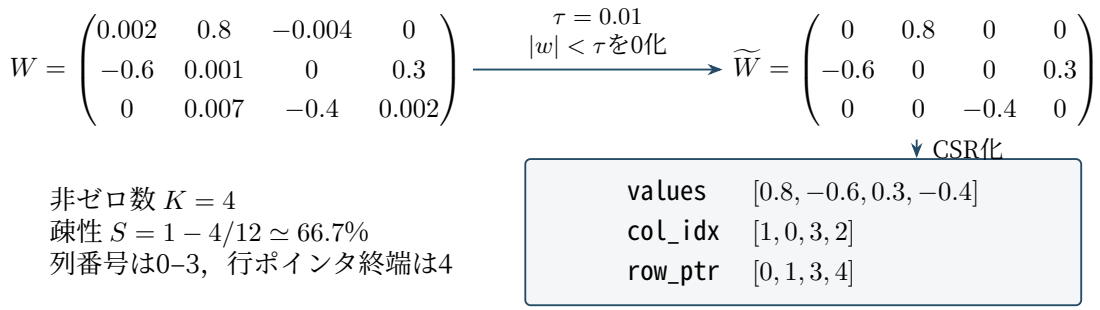


図 2: 枝刈りからCSRへの変換例．0化した密配列を，非ゼロ値・列番号・行ポインタへ変換する．

CSR形式の行列ベクトル積は次のように実装する．

コード 1: CSR形式による1層分の行列ベクトル積（概略）

```
for (uint16_t row = 0; row < rows; ++row) {
    float sum = bias[row];
    for (uint16_t p = row_ptr[row]; p < row_ptr[row + 1]; ++p) {
        sum += values[p] * input[col_idx[p]];
    }
    output[row] = sum;
}
```

2.5 評価指標

評価用カラーチャートの色数をN，色iの正解RGB値を$\boldsymbol{t}_i$， 推論値を$\boldsymbol{y}_i$とすると，MSEを

$$\mathrm{MSE} = \frac{1}{3N} \sum_{i=1}^{N} \sum_{c \in \{R,G,B\}} (y_{i,c} - t_{i,c})^2 \tag{13}$$

とする．Nは実際に用いる評価チャートの色数であり，3×3の場合は$N = 9$である．ニューラルネットワークへの 入出力は0.0–1.0へ正規化する一方，最終評価では従来どおり0–255のRGB値へ戻して P3形式のPPM画像を生成し，配布済みの`mse_calculator.html`で 評価チャートと同じ寸法・色順の基準PPM画像と比較する．学習に使用したチャートは 最終評価に流用せず，基準画像の対応とRGB尺度を確認する．全条件で同じ変換方法を使用する． 10回測定したMSEの平均を$\overline{\mathrm{MSE}}$とし，圧縮前からの絶対差と相対変化率を それぞれ

$$D_{\mathrm{MSE}} = \overline{\mathrm{MSE}}_{\mathrm{compressed}} - \overline{\mathrm{MSE}}_{\mathrm{dense}} \tag{14}$$

および

$$\Delta_{\mathrm{MSE}} = \frac{D_{\mathrm{MSE}}}{\overline{\mathrm{MSE}}_{\mathrm{dense}}} \times 100\ [\%] \tag{15}$$

で表す．基準の平均MSEが0の場合は相対変化率を算出せず，絶対差で評価する． MSEの単位は0–255のRGB値に基づく階調値の二乗である． また，モデルデータ削減率R_Bと圧縮比C_Bを

$$R_B = \left(1 - \frac{B_{\mathrm{compressed}}}{B_{\mathrm{dense}}}\right) \times 100\ [\%], \tag{16}$$

$$C_B = \frac{B_{\mathrm{dense}}}{B_{\mathrm{compressed}}} \tag{17}$$

とする．

3 実験方法

3.1 実験条件

独立変数，従属変数，統制条件を表2に示す.

表 2: 実験変数

分類	内容
独立変数	隠れ層次元H（40，50，60），圧縮の有無（Dense-FP32／枝刈り+CSR），枝刈り閾値τ
従属変数	MSE，MSE絶対差・相対変化率，疎性，モデルデータ量，削減率，コンパイル・書き込み・実行可否，RAM・フラッシュ使用量，推論時間
統制条件	学習データ，学習済み重み，評価チャート，黒・白校正手順，`SAMPLE_COUNT`，`RGB_FLASH_DELAY`，RGB尺度，測定治具，照明，チャート位置，紙の圧着，ソフトウェア版

使用機材と実行環境を表3に，学習・圧縮・測定の設定を 表4にまとめる．メモリ容量は推論を実行する主マイコンRA4M1の公称値である． 実際に利用できる容量と配列の配置先は，ボードコア・ビルド設定の影響を含めて確認する.

表 3: 使用機材・実行環境とArduinoのメモリ仕様

項目	仕様・設定	実験での扱い
ボード	Arduino UNO R4 WiFi	圧縮前後で同一の実機を使用
主マイコン	Renesas RA4M1	ニューラルネットワークの推論を実行
SRAM	32 KB（32,768 byte）	静的領域・作業配列・スタック等が使用
フラッシュメモリ	256 KB（262,144 byte）	プログラムと配置された定数データ等が使用
補助マイコン	ESP32-S3	推論モデルの格納先に含めない
読取機材	授業で使用するセンサ・光源，固定治具，遮光部材	配置・照明・紙の固定条件を統一
学習用PC	PCのCPU上で学習	同じ学習データから各Hの重みを作成
Arduino環境	Arduino IDE，ボードコア，コンパイラ	バージョンを実験開始時に記録し，以後固定
Python環境	Python，NumPy，PyTorch	バージョンを実験開始時に記録し，以後固定

表 4: 学習・圧縮・測定の設定値

項目	設定値
ネットワーク構成	入力3，隠れ層H・H，出力3（隠れ層のみReLU）
基本の隠れ層次元	$H = 40, 50, 60$
損失関数／最適化手法	MSE／Adam
学習率／L1正則化係数	0.01／10^{-6}
学習回数／乱数シード	50,000 epoch／42（NumPy・PyTorch）
枝刈り閾値の候補	$\tau \in \{0.001, 0.005, 0.01, 0.02, 0.05\}$（検証用データで選択）
センサ平均回数	`SAMPLE_COUNT`=10回
RGB読取待機時間	`RGB_FLASH_DELAY`=10 ms
光源の安定待機時間	測定前に60秒間点灯
MSEの反復測定	各Hで2条件を各10回（計6条件・60回）
推論時間の測定	10回ウォームアップ後，100回連続実行を5組

授業資料の60次元で実行可，70次元でコンパイル不可という結果は参考とし， 使用する実機でもDense-FP32の搭載限界を確認する．

学習条件は表4のとおりとする．圧縮の影響だけを比較するため，各Hについて一度得た 同一の学習済み重みからすべての圧縮条件を生成し，圧縮条件ごとに再学習は行わない． 学習用データは，3×3の9色と，6×6で読み取った画像の上4行 （6×4の24色）を合わせた33組とする． RGBを0–255から0.0–1.0へ正規化して`train_L1_normalization.ipynb`へ入力する． 教師データは対応する基準画像から各色のRGB値を取得する．縮小には最近傍法を用い， 6×4の読取データには6×6の基準画像の上4行を対応させるなど，色の位置を一致させる． 読取ミスを学習しないよう，元のPPM画像を目視確認してから学習データへ変換する． モデル生成前にNumPyとPyTorchへ表4の乱数シードを設定する． データの並び順，学習済み重み，学習ログを保存する．次元ごとに1回の学習とし， 本実験の結論はその学習済みモデルに対する圧縮の影響に限定する．

3.2 学習・検証・最終評価データの分離

データの用途は次の3つに分け，ファイル名とチャート識別子で管理する．

1. **学習用：**上記33組を重みの学習に使用する．
2. **検証用：**学習に使用していない色を含む別チャートの読取値と正解RGB値を用意し，枝刈り閾値の選択に使用する．読取は独立した3回分を取得し，PC上の事前評価に用いる．
3. **最終評価用：**学習用・検証用と別のチャートを用い，可能であれば教員から配布を受ける． 各隠れ層次元の枝刈り閾値を固定した後の実機評価だけに使用する．

検証用と最終評価用には，同じチャートを単に再読取したデータを用いず，色の組合せが異なるものを用いる． 別々のチャートを用意できない場合は，学習に用いない1枚の追加チャートの色を， 測定結果を見る前に検証用と最終評価用へ分割し，対応する色だけでMSEを計算する． その場合は色数，分割方法，同一チャート内の分割であるという制約を記録する． 現時点では追加チャートの入手可否は未確定のため，実験開始前に用意できるチャートと分割方法を確定する． 独立した検証・最終評価用の色を用意できない場合でも，実装・固定入力比較・容量・速度の確認を先行する． ただし，学習データでのMSEは参考値とし，独立データ

での精度確認が完了したとは扱わない． 最終評価結果を見て同じ評価データで閾値を選び直すことはしない．

3.3 実験条件の一覧

表 5: 実機で比較する条件

条件	重み表現	バイアス	目的
Dense-FP32	FP32密配列	FP32	圧縮前の基準
Prune-CSR	閾値枝刈り後のFP32値とCSR索引	FP32	小さい重みを除去した疎行列圧縮

まずPC上で検証用データに対して全閾値を事前評価し，Arduino実機へ載せるPrune-CSR条件を 各Hにつき1条件へ絞る．Dense-FP32と選んだPrune-CSRの2条件を， 40，50，60次元でそれぞれ比較するため，実機測定は基本6条件となる． 実装確認用として，PC上では枝刈り後のFP32密配列（Pruned-Dense）も生成する． これは枝刈りによる誤差とCSR実装による誤差を切り分けるために用い，実機の測定条件には含めない． 授業課題として優先する圧縮前後のMSE比較を，容量・速度の測定と合わせて実施する． 40，50，60次元の比較が完了した後，70次元のDense-FP32と圧縮版で搭載可否を比較し， 容量に余裕があれば100次元を試す．200次元は発展課題とし，必須条件の測定を優先する．

学習から実機評価までの流れを図3に示す． 検証用データではDense-FP32も同じ入力で評価し，MSE増加率の基準とする． 最終評価用データは閾値決定後に使用し，閾値の選択へ戻さない．

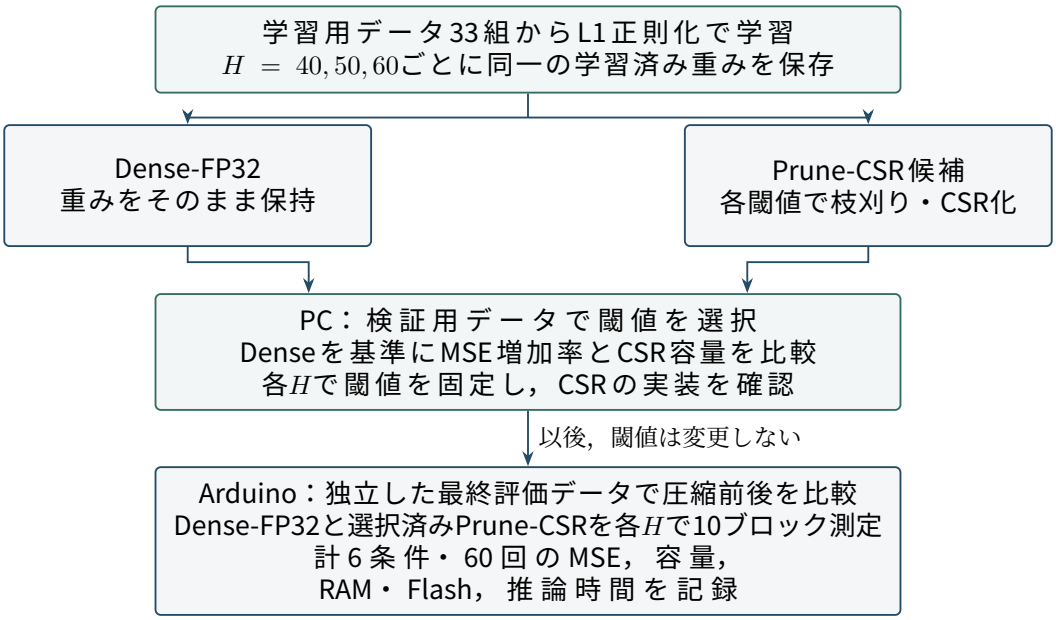


図 3: 実験全体の流れ．学習用・検証用・最終評価用データを分離する．実センサでの反復測定に加え，保存済み入力による固定入力比較も行う．

3.4 プログラムの実装手順

1. **基準モデルの作成：** $H = 40, 50, 60$について，3×3および6×4チャートから取得した 同一の正規化済み学習データを用いて学習する． Notebook側では単一の変数Hを3つのLinear層へ使用し，Arduino側の HIDDEN_SIZE1とHIDDEN_SIZE2も同じ値へ変更する．学習用・検証用・最終評価用のデータを分離し，最終評価用は選択段階で使用しない．

2. **PC上での圧縮候補生成：** Pythonで各重みに閾値を適用し，疎性，CSR理論容量，検証用データでの圧縮後MSEを計算する． PC上でもArduinoと同じ前処理・後処理を適用し，0–255のRGB値でMSEを求める． 枝刈り後の重みをCSRへ変換し，検証結果に基づいて実機比較に用いる閾値を決める．

3. **ヘッダファイルの出力：** 学習ノートブックが出力するmodel_parameters.hを基準とする． Dense-FP32では従来のweight_1[]–weight_3[]と bias_1[]–bias_3[]を出力する．Prune-CSRでは各層ごとに values[]，col_idx[]，row_ptr[]の3配列を出力し， それぞれ非ゼロ値，列番号，行開始位置を格納する．

   密形式では，重み要素数が$3H$，H^2，$3H$であることを検査する． 圧縮前後とも，バイアス要素数がH，H，3であることを確認する． CSRでは第l層の非ゼロ数をK_l，行数をm_lとして，値・列番号の配列長がK_l， 行ポインタの長さが$m_l + 1$であることを確認する．各層の配列には層番号を付け，名前の重複を避ける．

4. **Arduino推論処理の実装：** 密行列とCSRの行列ベクトル積関数を分離して実装する．重みを出力ニューロン単位の行優先で参照し， 隠れ層1・2だけにReLUを適用する．出力層にはReLUを適用しない． normalizeInput()では，入力を (raw − black)/(white − black)で正規化して0–1へクリップする． outputTo255()では，出力を0–1へクリップし，255倍して四捨五入する． 黒・白校正値が同一で分母が0になる場合は校正不成立とし，再校正する． 圧縮前後で，この前処理・後処理とバイアス加算順序は変更しない．配布プログラムの仕様も実装前に確認し，変更点を実験記録へ残す．

5. **単体確認：** PC上で枝刈り後の密行列とCSRから復元した行列が一致することを確認する． 同じ固定入力に対するPruned-DenseとPrune-CSRの出力を比較した後，PC版とArduino版の各層出力を比較する． CSRの行ポインタ先頭が0，終端がK_l，並びが非減少であること，列番号が列数未満であること， 復元後の行列形状，整数のオーバーフロー，添字範囲を確認する． 行列の復元一致と，浮動小数点演算による出力の近似一致を区別する． 各層出力は絶対誤差10^{-5}と相対誤差10^{-4}を目安に照合し，超過時は演算順と実装を調べる．

6. **ビルド・実行確認：** 各条件を同一ボード設定でクリーンビルドし，コンパイル可否，フラッシュ・静的RAM使用量を記録する． 書き込み後，固定入力に対する出力と推論時間を確認する．

3.5 性能確認方法

実装後の性能は，Arduinoへの搭載可否，フラッシュ・RAM使用量，推論時間，MSEによって確認する． 各Hの同一学習済み重みから作成したDense-FP32を圧縮前の基準とし，Prune-CSRを同一条件で比較する． ニューラルネットワークを用いないbatansyusei.inoは参考条件として区別し，圧縮前の基準には用いない． 特にMSE測定では，照明，紙の圧着，位置ずれ，周辺光による変動を圧縮誤差と混同しないため， 次の手順を用いる．

1. 学習に使用していない同一の評価用カラーチャートを全条件で使用する．

2. 配布の読取プログラムを基に，黒，白の順に校正してから各画素を読み取る． 使用するボタン割当ては配布コードで確認し，全方式で統一する． 書き込み後は毎回同じ位置・手順で校正し，得られた値と時刻を記録する．

3. 1色のセンサ値はSAMPLE_COUNT=10回の平均とし，RGBの読取時間ずれを抑えるため，RGB_FLASH_DELAY=10 msに固定する． センサ，光源，遮光，紙の固定治具，チャート位置を変えず，測定前に光源を60秒間点灯して安定させる．

4. 評価チャートのN色を同じ順序で読み取り，各回についてチャートと同じ寸法のP3形式PPM画像を保存する． mse_calculator.htmlへ評価チャートに対応する基準PPM画像と読取画像を入力してMSEを記録する． 色の一部を最終評価用に分けた場合は，指定した色のRGB値を抽出して同じMSEの式で計算する．

5. 各HについてDense-FP32とPrune-CSRを1回ずつ測ることを1ブロックとし，10ブロック実施する． 奇数ブロックはDense–CSR，偶数ブロックはCSR–Denseの順とし，時間変化と測定順序の偏りを抑える． 基本6条件，各10回の合計60回分の測定を行う．

6. 各回のMSEを保存し，平均，標本標準偏差（分母$n-1$），最小値，最大値を求める． 同一ブロック内の圧縮版とDense版のMSE差についても平均と標準偏差を求める． 測定ばらつきだけから有意差があるとは断定せず，差の大きさと変動を併記する．

物理的な読取誤差と圧縮誤差を分けるため，最終評価用のセンサ生値と黒・白校正値を保存し， 同じ保存済み入力を各方式へ与える固定入力比較も行う． これは閾値決定後の確認とし，この結果で閾値を選び直さない．

推論時間はmicros()で計測する．同じ保存済み入力列について10回のウォームアップ後，100回連続実行し，合計時間を100で割る．これを5組繰り返し，1推論当たりの時間の平均と標準偏差を求める． 計測区間は正規化済み入力からネットワーク出力までとし，CSRの索引参照，積和演算，ReLUを含める． センサ読取，待機，RGB画像への変換，シリアル出力は除外する． 出力のチェックサムを計測後に表示し，コンパイラによる未使用演算の削除を防ぐ．

3.6 閾値の選択とCSRの採用条件

PC上の閾値選択では，検証用データの平均MSE増加率が5%以下で，CSR容量が密形式未満となる候補から， CSR容量が最小の閾値を選ぶ．Denseの検証MSEが0の場合は，圧縮後も0であることを条件とする． 同容量なら検証MSEが小さい候補を選ぶ．該当候補がなければ CSRは採用条件を満たさないと記録し，容量が最小となる閾値を診断用に実機比較する． 実機比較を終えた後，各Hにおける枝刈り+CSRの採否は次の基準で判断する．

1. **必須条件：**Arduinoでコンパイル，書き込み，推論が正常に完了する．

2. **品質条件：**暫定基準として，圧縮前に対する平均MSEの増加率を5%以下とする． 基準の平均MSEが0の場合は，圧縮後も0となることを条件とする． さらに絶対差と反復測定のばらつきを併記する．

3. **容量条件：**CSR索引を含む実測モデル容量がDense-FP32未満であることとする． 静的RAM・フラッシュ使用量と作業配列容量も確認し，実機に搭載できることを条件とする．

4. **速度条件：**推論時間がDense-FP32より20%を超えて増加する場合は，容量削減との利得を比較し，採否を再検討する．

5. **実装条件：**枝刈り後の密行列とCSRから復元した行列が一致し， 行ポインタと列番号の検査，固定入力による出力比較を完了していることとする．

5%および20%は本計画で比較を行うための暫定基準であり，授業で指定された値ではない． PC上では閾値別の検証MSEとCSR容量を図示し，精度と容量のトレードオフを確認す

る． 実機では各HのDense-FP32とPrune-CSRのMSE・容量・速度を比較し， 有効な条件と制約を整理する．採用条件を満たす閾値が得られない場合や， 最終評価で品質条件を満たさない場合も，失敗した条件とその結果を含めて報告する． やむを得ず閾値を再調整する場合は，当該データを検証用として扱い，新しい最終評価データを用意する．

3.7 結果の記録様式

各条件について，表6と表7を同じ条件IDで対応させて記録する． 条件IDにはH，方式，閾値，乱数シードを含める．結果欄は測定後に記入し， コンパイル・書き込み・実行はそれぞれ可／不可，未実施は未実施と記録する．

表 6: 容量・メモリ使用量の記録表

H	方式	τ	疎性 [%]	モデル [byte]	削減率 [%]	静的RAM [byte]	Flash [byte]
40	Dense	0					
40	CSR						
50	Dense	0					
50	CSR						
60	Dense	0					
60	CSR						

表 7: 動作・品質・速度の記録表

H	方式	コンパイル／書込／実行	平均MSE $\pm s$	D_{MSE}	Δ_{MSE} [%]	推論時間 平均$\pm s$ [μs]
40	Dense					
40	CSR					
50	Dense					
50	CSR					
60	Dense					
60	CSR					

各回のMSE，最小値・最大値，同一ブロック内のMSE差，圧縮比，作業配列容量， 時間計測5組の生データと，PC上で評価した全閾値の検証結果は別の測定CSVへ保存する． ビルドや書き込みに失敗した場合も行を削除せず，失敗段階とログを残す．

参考文献

[1] R. Tibshirani, “Regression Shrinkage and Selection via the Lasso,” Journal of the Royal Statistical Society: Series B, vol. 58, no. 1, pp. 267–288, 1996.

[2] S. Han, J. Pool, J. Tran, and W. J. Dally, “Learning Both Weights and Connections for Efficient Neural Networks,” Advances in Neural Information Processing Systems 28, pp. 1135–1143, 2015.

[3] N. Bell and M. Garland, “Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors,” Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis, Article 18, pp. 1–11, 2009.